WELT 5 · DS 35 (FÜR SCHNELLE) UND VORBEREITUNG AUF BOSS 5 (DS 36)

# W5·3 – Transfer: Unbekannte Bäume erschließen

Papier first – kein neues IDE-Level · Papier-Compiler mit der Testumgebung „Runenbuch“ (gibt die Lehrkraft aus) · Mini-Abi mit 18 BE · Im Lab (Algo Quest Lab): Level 5.6

Am Ende des Waldes warten die **Baumhüter**. Jeder hütet eine Baumart, die du noch nie gesehen hast – genau wie im Abitur: Dort stand 2024 ein B-Baum im Material, 2025 ein Huffman-Baum, 2026 ein Heap. Niemand erwartet, dass du diese Bäume kennst. Erwartet wird, dass du eine **unbekannte Struktur** aus dem Material erschließt und dein Wissen über Binärbäume, Rekursion und Suchbäume darauf überträgst.

Dafür brauchst du eine **Strategie**. Du übst sie heute an einer vierten Baumart, die im Boss **nicht** vorkommt: dem **Präfixbaum** (englisch *Trie*). Die Waldhexe speichert damit ihre Zauberwörter – im **Runenbaum**.

### Die Fünf-Schritte-Strategie für unbekannte Bäume

1. **Definition lesen:** Markiere im Material: Was steht in einem Knoten? Was bedeuten die Kanten (links/rechts, Kinder)? Welche Regeln werden genannt? Welche Begriffe sind neu?
2. **Beispiel von Hand:** Vollziehe das Beispiel aus dem Material nach (Wörter ablesen, Wert berechnen, Weg verfolgen) und spiele eine kleine Änderung selbst durch (einfügen, suchen).
3. **Invariante formulieren:** Was gilt **immer** – vor und nach jeder Operation? Aus der Invariante folgen Such- und Prüfverfahren und viele Begründungen.
4. **Auf BinTree abbilden:** Welcher Inhaltstyp (Cast!)? Was bedeuten getLeft und getRight? Was ist der leere Baum? Wo endet ein Weg?
5. **Implementieren:** Bauplan rekursiver Baummethoden (Basisfall leer – Wurzel auswerten – Teilbäume rekursiv – verknüpfen) und Abi-Check.

**So hast du es schon gemacht:** Jede Baumart in Welt 5 lässt sich mit denselben Fragen beschreiben.

| Baumart | Inhalt eines Knotens | links / rechts | Invariante (gilt immer) |
|---|---|---|---|
| Termbaum | Rechenzeichen oder Zahl (Zeichenkette) | linker / rechter Operand | Blätter sind Zahlen, innere Knoten Rechenzeichen mit genau zwei Teilbäumen |
| Akinator | Frage oder Monstername (Zeichenkette) | Antwort ja / nein | Blätter sind Monster, innere Knoten Fragen; Monster = Fragen + 1 |
| Suchbaum | Ganzzahl bzw. Item (Schlüssel) | kleinere / größere Schlüssel | für jeden Knoten: links alle kleiner, rechts alle größer |

### Operatoren im Abitur (Auswahl) – was wird verlangt?

| Operator | Was du tun musst |
|---|---|
| angeben, nennen | Ergebnis oder Aussage ohne Begründung hinschreiben (z. B. Wörter, Rückgabe, Baum in Klammerform) |
| beschreiben | Sachverhalt oder Ablauf strukturiert in eigenen Worten wiedergeben |
| darstellen | Struktur oder Zustand übersichtlich wiedergeben – bei Bäumen: zeichnen |
| bestimmen | Ergebnis durch Durchführen eines Verfahrens ermitteln; der Weg muss erkennbar sein |
| dokumentieren | Ablauf schrittweise festhalten, meist in einer Tabelle (Trace- oder Rekursionstabelle) |
| erläutern | Sachverhalt nachvollziehbar machen – mit Begründungen und Beispielen |
| begründen | Aussage auf Regeln, Definitionen oder Invarianten zurückführen |
| analysieren | unbekannten Code oder unbekannte Struktur untersuchen: Was passiert, was wird berechnet? |
| implementieren, modifizieren | in Java umsetzen bzw. gegebenen Code gezielt verändern (Signatur wie gefordert!) |
| beurteilen | zu einem begründeten eigenen Urteil kommen – Kriterien nennen, abwägen, Fazit |

## Teil 1 – Die Strategie am Runenbaum

### Material M1: Präfixbaum

Ein **Präfixbaum** speichert eine Menge von Wörtern aus Großbuchstaben. Seine **Wurzel enthält keinen Buchstaben**, jeder andere Knoten genau einen. Ein Knoten kann **beliebig viele Kinder** haben; die Kinder eines Knotens enthalten verschiedene Buchstaben und sind von links nach rechts **alphabetisch geordnet**.

Liest man die Buchstaben auf dem Weg von der Wurzel zu einem Knoten, erhält man einen **Wortanfang (Präfix)**. Ist der Knoten **markiert** (doppelter Kreis), ist dieser Wortanfang ein gespeichertes Wort. Wörter mit gleichem Anfang teilen sich die Knoten dieses Anfangs.

**Einfügen** eines Worts: Von der Wurzel aus wird für jeden Buchstaben das Kind mit diesem Buchstaben gesucht und betreten; fehlt es, wird es an der alphabetisch richtigen Stelle neu angelegt. Der Knoten des letzten Buchstabens wird markiert.

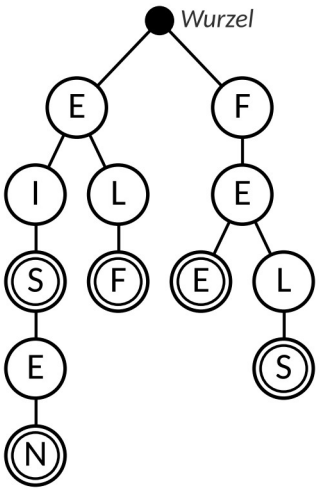



*Abbildung M1: Der Runenbaum der Waldhexe als Präfixbaum*

## Material M2: Darstellung als Binärbaum

Um einen Präfixbaum mit der Klasse BinTree zu speichern, wird jeder Knoten außer der Wurzel zu einem Knoten eines Binärbaums vom Inhaltstyp **Rune** (Abbildung M2b). Es gilt die **Erstes-Kind/Nächster-Geschwister-Darstellung**:

- Der **linke** Teilbaum eines Knotens (Kante **K**) beginnt mit seinem **ersten Kind** (dem alphabetisch kleinsten).
- Der **rechte** Teilbaum eines Knotens (Kante **G**) beginnt mit seinem **nächsten Geschwister** (dem nächsten Kind desselben Elternknotens).
- Der Binärbaum beginnt mit dem ersten Kind der Wurzel. Ein leerer Binärbaum steht für einen Präfixbaum ohne Wörter.

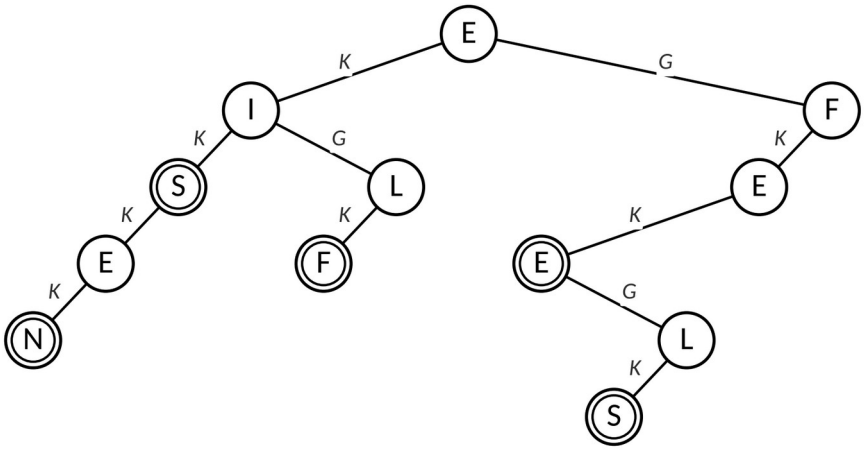



*Abbildung M2a: Der Runenbaum aus M1 als Binärbaum (doppelter Kreis: istEnde() liefert wahr)*

| **Rune** |
|---|
| - zeichen: Zeichen<br>- ende: Wahrheitswert |
| + c Rune(zeichen: Zeichen, ende: Wahrheitswert)<br>+ getZeichen(): Zeichen<br>+ istEnde(): Wahrheitswert<br>+ setEnde(e: Wahrheitswert): nichts |

*Abbildung M2b: Klassenkarte Rune*

### Material M3: Die Operation enthaelt

`enthaelt(b: BinTree vom Inhaltstyp Rune, wort: Zeichenkette): Wahrheitswert` liefert wahr, wenn wort (nicht leer) im Runenbaum b gespeichert ist.

```
1  public static boolean enthaelt(BinTree b, String wort) {
2      if (b.isEmpty()) {
3          return false;                        // Buchstabe fehlt
4      }
5      Rune r = (Rune) b.getItem();
6      if (r.getZeichen() != wort.charAt(0)) {
7          return enthaelt(b.getRight(), wort); // bei den Geschwistern suchen
8      }
9      if (wort.length() == 1) {
10         return r.istEnde();                  // letzter Buchstabe: Wortende?
11     }
12     String rest = wort.substring(1, wort.length());
13     return enthaelt(b.getLeft(), rest);      // nächster Buchstabe bei den Kindern
14 }
```

## Aufgabe 1: Schritt 1 und 2: Lesen und von Hand nachvollziehen

PAPIER ★

**a)** Gib alle Wörter an, die im Runenbaum aus Abbildung M1 gespeichert sind. **[1 BE]**

**b)** Erläutere am Beispiel EIS und EISEN, warum ein gespeichertes Wort nicht immer an einem Blatt endet und warum es deshalb die Markierung braucht. **[2 BE]**

**c)** Nacheinander werden ELFE, EI und ERZ eingefügt. Stelle den Präfixbaum danach dar und gib an, wie viele Knoten (ohne Wurzel) durch jedes der drei Wörter dazukommen. **[3 BE]**

**d)** Beschreibe, wie man mit dem Präfixbaum prüft, ob ein Wort gespeichert ist. Gib zwei verschiedene Gründe an, aus denen die Prüfung „nein“ ergeben kann. **[2 BE]**

## Aufgabe 2: Schritt 3: Was gilt immer?

PAPIER ★★

**a)** Formuliere drei Invarianten des Präfixbaums – Eigenschaften, die nach jedem Einfügen gelten. Eine davon soll sich auf die Blätter beziehen. **[3 BE]**

**b)** Begründe, dass die Anzahl der gespeicherten Wörter gleich der Anzahl der markierten Knoten ist. **[1 BE]**

**c)** Begründe, dass die Reihenfolge des Einfügens keinen Einfluss auf die Gestalt des Präfixbaums hat. Vergleiche mit dem binären Suchbaum (AB 2, Aufgabe 3). **[2 BE]**

## Aufgabe 3: Schritt 4: Auf BinTree abbilden

PAPIER ★★

**a)** t ist der Binärbaum aus Abbildung M2a. Gib für jeden Teilbaum das Zeichen seiner Wurzel und den Wert von istEnde() an – z. B. für die erste Zeile die Werte von `((Rune) t.getLeft().getItem()).getZeichen()` und `((Rune) t.getLeft().getItem()).istEnde()`. **[3 BE]**

| Teilbaum | getZeichen() | istEnde() |
|---|---|---|
| `t.getLeft()` | | |
| `t.getRight()` | | |
| `t.getLeft().getLeft()` | | |
| `t.getLeft().getRight()` | | |
| `t.getRight().getLeft().getLeft().getRight()` | | |

**b)** Ergänze die Tabelle „So hast du es schon gemacht“ um eine Zeile für den Runenbaum in der Darstellung aus M2. **[2 BE]**

| Baumart | Inhalt eines Knotens | links / rechts | Invariante (gilt immer) |
|---|---|---|---|
| Runenbaum | | | |

**c)** Die Wörter NEBEL, NEST und NOX werden in einen leeren Runenbaum eingefügt. Stelle das Ergebnis als Präfixbaum und als Binärbaum in der Darstellung aus M2 dar. **[3 BE]**

**d)** Gib für den Binärbaum aus Abbildung M2a die Höhe und die Länge des längsten gespeicherten Worts an. Begründe allgemein, warum die Höhe des Binärbaums größer sein kann als die Länge des längsten Worts. **[2 BE]**

## Aufgabe 4: Schritt 5a: Fremden Code lesen

PAPIER ★★

**a)** Dokumentiere den Aufruf `enthaelt(t, "FELS")` (t aus Abbildung M2a) in einer Rekursionstabelle. In der Spalte „Fall“ notierst du, welcher Zweig von M3 ausgeführt wird; markierte Knoten schreibst du mit Stern (z. B. S*). **[4 BE]**

| Rekursionstiefe *) | Knoten | wort | Fall | Rückgabe |
|---|---|---|---|---|
| 0 | E | "FELS" | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

**) Bei jedem rekursiven Aufruf erhöht sich die Rekursionstiefe um 1. Der Erstaufruf hat die Rekursionstiefe 0.*

**b)** Gib für t die Rückgabe von enthaelt und die Anzahl der untersuchten (nicht leeren) Knoten an. **[2 BE]**

| wort | "EI" | "EISE" | "FELSEN" | "E" |
|---|---|---|---|---|
| Rückgabe | | | | |
| untersuchte Knoten | | | | |

**c)** Erläutere, welche Bedeutung der Aufruf in Zeile 7 und der Aufruf in Zeile 13 im Präfixbaum aus M1 haben. **[2 BE]**

## Aufgabe 5: Schritt 5b: Implementieren

PAPIER → IDE ★★

**a)** Implementiere die Methode `anzahlWoerter(b: BinTree vom Inhaltstyp Rune): Ganzzahl`, die die Anzahl der im Runenbaum b gespeicherten Wörter liefert (Abbildung M2a: 5, leerer Baum: 0). Beachte: Auch die Geschwister gehören dazu. **[4 BE]**

**b)** **Papier-Compiler:** Tippe deine Methode unverändert in die Testumgebung „Runenbuch“ (Klasse Runenbaum, Abschnitt DEIN TEIL) und starte im Testrunner `RunenbuchTest`. Der Test `papier_anzahlWoerter` muss grün sein.

### Regeln des Papier-Compilers

1. Schreibe die Methode **vollständig von Hand** – ohne Gerät, ohne Spickzettel außer den Werkzeugkarten.
2. Gehe den Abi-Check durch (Kasten unten).
3. Tippe deine Lösung **unverändert** ab – Zeichen für Zeichen, auch mit Fehlern!
4. Starte den Testrunner: `papier_anzahlWoerter` muss grün sein (`papier_istPraefix` gehört zum Mini-Abi).
5. Jeder Fehler kommt ins Fehlerlogbuch – erst dann korrigierst du in der IDE.

## Fehlerlogbuch

| Nr. | Was war falsch? | Art: Syntax / Logik / Abi-Check | So vermeide ich es |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |

### Abi-Check: Unbekannte Bäume

- ☐ Material vollständig gelesen? Definitionen markiert, Beispiel nachvollzogen?
- ☐ Was ist der Inhaltstyp – und wie wird gecastet? Was bedeuten links und rechts **in dieser** Struktur?
- ☐ Höhe, Ebene, Tiefe: Gilt eine Definition aus dem Material? Sie hat Vorrang vor unserer.
- ☐ Invariante genutzt? (Sortierung, Markierungen, Ordnung) – sie liefert Begründungen und Abbruchbedingungen.
- ☐ Basisfall `if (b.isEmpty())` zuerst; beide Teilbäume bedacht – auch wenn einer „anders“ gemeint ist (Geschwister!).
- ☐ Zeichenketten nur mit erlaubten Operationen: length(), charAt(i), substring(a, b), equals, compareTo.

## Teil 2 – Mini-Abi: Das Zauberbuch der Waldhüterin (18 BE, ca. 45 min)

*Die Teilaufgaben sind wie im Abitur formuliert (mit „Sie“). Hilfsmittel: nur die Materialien M1 bis M3 und die Werkzeugkarten. Schreibe alles von Hand.*

Die Waldhüterin speichert die Namen ihrer Zutaten in einem Runenbaum. Abbildung M4 zeigt ihn in der Darstellung aus Material M2.

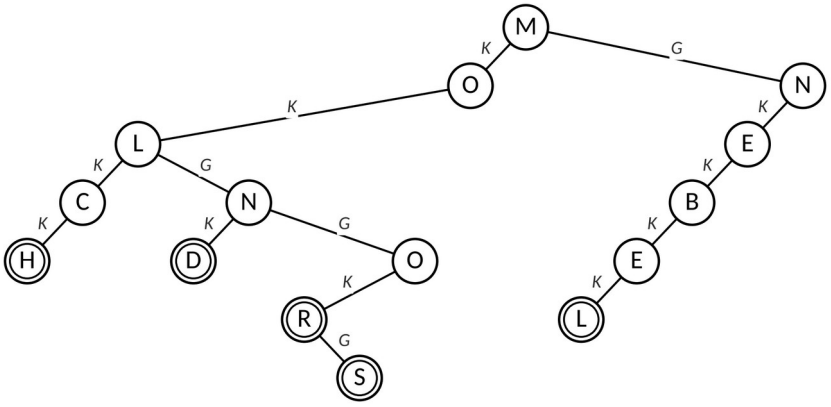



*Abbildung M4: Der Runenbaum der Waldhüterin (K = erstes Kind, G = nächster Geschwister; doppelter Kreis: Wortende)*

**a)** Geben Sie alle im Runenbaum aus Abbildung M4 gespeicherten Wörter in alphabetischer Reihenfolge an. **[2 BE]**

**b)** Die Wörter MODER und NEST werden in dieser Reihenfolge eingefügt. Stellen Sie den Binärbaum nach beiden Einfügungen dar. **[3 BE]**

**c)** Bestimmen Sie für den Runenbaum aus Abbildung M4 jeweils die Rückgabe von enthaelt (Material M3) und die Anzahl der untersuchten Knoten für die Wörter MOOS, MOO und MOHN. **[2 BE]**

Gegeben ist die folgende Methode:

```
1  public static int raetsel(BinTree b) {
2      if (b.isEmpty()) {
3          return 0;
4      }
5      int x = 1 + raetsel(b.getLeft());
6      int y = raetsel(b.getRight());
7      if (x > y) {
8          return x;
9      }
10     return y;
11 }
```

**d)** Analysieren Sie die Methode raetsel: Bestimmen Sie `raetsel(t)` für den Runenbaum t aus Abbildung M4 und beschreiben Sie, was raetsel für einen Runenbaum allgemein berechnet. Erläutern Sie, warum nur beim linken Teilbaum 1 addiert wird. **[4 BE]**

**e)** Implementieren Sie die Methode `istPraefix(b: BinTree vom Inhaltstyp Rune, p: Zeichenkette): Wahrheitswert`. Sie liefert wahr, wenn mindestens ein gespeichertes Wort mit p beginnt (p ist nicht leer; ein Wort beginnt auch mit sich selbst). Für Abbildung M4 gilt z. B. istPraefix(t, "MOO") = wahr und istPraefix(t, "MI") = falsch. **[4 BE]**

**f)** Ein Mitglied Ihres Teams schlägt vor, enthaelt (Material M3) so zu ändern, dass die Suche bei den Geschwistern sofort mit falsch abbricht, sobald der gesuchte Buchstabe **kleiner** ist als der Buchstabe des aktuellen Knotens. Beurteilen Sie den Vorschlag hinsichtlich Korrektheit und Aufwand. **[3 BE]**

**Papier-Compiler zu e):** Tippe istPraefix unverändert in die Testumgebung „Runenbuch“ – `papier_istPraefix` muss grün sein.

> 🎯 **Im Lab üben: Level 5.6** „Fremde Bäume – Transfer auf unbekannte Baumarten“ – Punkte gibt es dort automatisch. Zur Vorbereitung auf Boss 5 wartet dort außerdem der **Online-Boss** „Die drei Baumhüter – Online-Duell“ (25 min, bester Versuch zählt).

### Für Schnelle

- Schreibe `woerter(b: BinTree vom Inhaltstyp Rune, praefix: Zeichenkette): Zeichenkette`, die alle gespeicherten Wörter mit Leerzeichen getrennt liefert. Welche Traversierung liefert sie alphabetisch?
- Ein Runenbaum enthält **alle** Wörter aus drei Buchstaben von AAA bis ZZZ. Wie viele Wörter sind das, und wie viele Knoten untersucht enthaelt höchstens? Vergleiche mit einem ausgeglichenen Suchbaum für dieselben Wörter.